

EXPERIMENT - 42

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

- Remove all vowels from a string.
- Find the first repeating lowercase alphabet in a string.
- Check if two strings are anagrams of each other.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu and read choice.

Step 3: Using a switch statement on choice:

 If choice is 1, read a string, call `removeVowels()` and display the result.

 If choice is 2, read a string, call `firstRepeatingLowercase()`, and display the letter it returns, or a message if it returns `'\0'`.

 If choice is 3, read two strings, call `areAnagrams()` and display whether they are anagrams.

 If choice is 4, display "Exiting the program."

Step 4: Repeat from Step 2 while choice is not equal to 4.

Step 5: STOP

Function `removeVowels()`

Step 1: Set `j = 0` and repeat for `i = 0` while `s[i]` is not `'\0'`:

 If `s[i]` is not one of a, e, i, o, u, A, E, I, O or U, copy it into `result[j]` and add 1 to `j`.

Step 2: Put `'\0'` at `result[j]`.

Step 3: Return to the calling function.

Function `firstRepeatingLowercase()`

Step 1: Set all 26 elements of `count` to 0.

Step 2: Repeat for `i = 0` while `s[i]` is not `'\0'`: if `s[i]` lies between 'a' and 'z', add 1 to `count[s[i] - 'a']`.

Step 3: Repeat for `i = 0` while `s[i]` is not `'\0'`: if `s[i]` lies between 'a' and 'z' and `count[s[i] - 'a']` is greater than 1, return `s[i]`.

Step 4: Return `'\0'`, since no lowercase letter repeats.

Function `areAnagrams()`

Step 1: Set all 256 elements of `count1` and `count2` to 0, and set `len1 = 0` and `len2 = 0`.

Step 2: Repeat for every character of `s1`: add 1 to `count1` at that character and add 1 to `len1`.

Step 3: Repeat for every character of `s2`: add 1 to `count2` at that character and add 1 to `len2`.

Step 4: If `len1` is not equal to `len2`, return 0.

Step 5: Repeat for $i = 0$ to 255: if $\text{count1}[i]$ is not equal to $\text{count2}[i]$, return 0.

Step 6: Return 1.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Remove vowels	Programming in C	Prgrmmng n C, with the spaces kept	Prgrmmng n C, both spaces still there	PASS
2	1. Remove vowels	Beautiful Day	Btfl Dy	Btfl Dy, so a capital B survives but a capital A would not	PASS
3	2. First repeating	programming	r, because r appears before g and m do	r, not g, so the rule is working	PASS
4	2. First repeating	AABBcdc	c, since A and B are capitals and do not count	c, the capitals were skipped	PASS
5	2. First repeating	xyz	A message, as nothing repeats	Said no lowercase letter repeats	PASS
6	3. Anagram	listen and silent	They are anagrams	Said they are anagrams	PASS
7	3. Anagram	hello and world	Not anagrams	Said they are not	PASS
8	3. Anagram	Listen and silent	Not anagrams, because L and l are different characters	Said they are not, so the check is case sensitive	PASS
9	4. Exit	choice = 4	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

strops42.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\\0')
                    printf("No lowercase letter repeats in this string\\n");
                else
                    printf("First repeating lowercase letter: %c\\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\\n");
                else
                    printf("The two strings are not anagrams\\n");
                break;

            case 4:
                printf("Exiting the program.\\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```

/* i walks through the old string and j marks where to write in the new
one, and they drift apart because j only moves when something is
actually copied.
The '\0' on the last line is the one I forgot. Programming in C came
out right and then had junk stuck on the end of it, because printf
had no idea where the new string finished.
Both cases are listed because A and a are different characters, and
leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])

```

```

{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

```

```

/* First repeating can be read two ways, and the two give different
answers. In abcdca one reading says b, the first letter you meet
that you have seen before. The other says a, because out of the
letters that repeat at all, a is the one that showed up earliest.
I went with the second, which is the usual meaning, and that is why
the counting happens first and the walk from the left happens after.
One single pass would have given the other answer.
count has 26 boxes, one per small letter, and s[i] - 'a' turns a
letter straight into its box number, since 'a' is 97 and 'b' is 98.
Capitals are skipped in both loops, so AABBCdc answers c, even though
A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])

```

```

{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
found. */
    return '\0';
}

```

```

/* Anagrams hold the same characters the same number of times, only in
a different order. My first idea was to sort both strings and see if
they came out identical, but counting turned out to be less code, and
it leaves the two strings alone instead of rearranging them.
256 boxes covers every character there is, so spaces and digits are
counted along with the letters, and nothing is quietly ignored.
The length test is not only a shortcut. The counts could agree on
every character and the strings still be different lengths if one of
them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])

```

```

{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
    }

```

```
}

for (i = 0; s2[i] != '\0'; i++) {
    count2[(unsigned char) s2[i]]++;
    len2++;
}

if (len1 != len2)
    return 0;

for (i = 0; i < 256; i++)
    if (count1[i] != count2[i])
        return 0;

return 1;
}
```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra strops42.c -o strops42`, because those two flags show up even the small warnings, and it built clean.

Removing the Vowels, and the First Repeating Letter

```
adilshakil@ADILs-MacBook-Air ~ % gcc -Wall -Wextra strops42.c -o strops42
adilshakil@ADILs-MacBook-Air ~ % ./strops42

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 1
Enter a string: Programming in C
String without vowels: Prgrmmng n C

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 1
Enter a string: Beautiful Day
String without vowels: Btfl Dy

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 2
Enter a string: programming
First repeating lowercase letter: r
```

Capitals Skipped, Nothing Repeating, and a Pair of Anagrams

```
adilshakil — -zsh — 120x30

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 2
Enter a string: AABBBcdc
First repeating lowercase letter: c

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 2
Enter a string: xyz
No lowercase letter repeats in this string

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 3
Enter the first string: listen
Enter the second string: silent
The two strings are anagrams
```

Two Strings That Are Not Anagrams, and Exit

```
adilshakil — -zsh — 120x30

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 3
Enter the first string: hello
Enter the second string: world
The two strings are not anagrams

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 3
Enter the first string: Listen
Enter the second string: silent
The two strings are not anagrams

MENU
1. Remove all the vowels from a string
2. Find the first repeating lowercase letter
3. Check whether two strings are anagrams
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All nine cases came out right.

The one that made me stop and think was option 2, and it was not the code, it was the question. First repeating can be read two ways and they do not agree. Take abcdabca. One reading gives b, because b is the first letter you come across that you have already seen. The other gives a, because out of the letters that repeat at all, a

is the one that appeared earliest. I went with the second, which is how first repeating is usually meant, and that decided the shape of the function. It counts the whole string first and only then walks it from the left looking for a count above 1. Had I done it in a single pass, checking each letter against the ones behind it, I would have got the other answer instead.

The counting itself uses a trick I had not seen before this week. `count` has 26 boxes and `s[i] - 'a'` turns a letter straight into its box number, because 'a' is 97 and 'b' is 98, so subtracting 'a' leaves 0 and 1. No comparing every letter against every other letter. Capitals are left out of both loops on purpose, and that is what case 4 in the table is really testing: AABBCdc answers c, even though A and B repeat more times than c does.

In `removeVowels` I forgot the `result[j] = '\0'` on my first attempt. Programming in C printed correctly and then had a few stray characters hanging off the end, which confused me for a while because the part I could read was right. Nothing in the loop writes that '\0', so `printf` had no idea where the new string stopped and just kept going.

For the anagrams my first thought was to sort both strings and see whether they came out identical. Counting what is in each one turned out to be less code and it leaves the two strings where they are instead of shuffling them about. The length test at the top is not only there to save time either. The counts could agree on every single character and the strings still be different lengths, if one of them held something the other did not.

Case 8 is the one I would point at if anyone asked. Listen and silent look like anagrams when you read them, but the program says they are not, and that is correct, because in C the capital L is character 76 and the small l is 108. The check compares characters, not the letters as we think of them.

Last thing. The question lists three operations and no exit, but a menu built on a do while loop has nothing to stop it, so option 4 is mine. Without it the menu simply keeps coming back and the only way out is to kill the program from the terminal.